

Desenvolvimento Web com .Net

Aula 1

Prof. Me. Lucas Teodoro dos Santos

UNIP

05 de Agosto de 2026

Roteiro da aula

- 1 Apresentação da Disciplina
- 2 O que é uma aplicação Web?
- 3 Front-end
- 4 Back-end
- 5 HTTP e HTTPS
- 6 Ecosistema .Net
- 7 MVC
- 8 Primeiro Projeto

Nossa dinâmica em sala

Para que a disciplina seja produtiva, a aprendizagem precisa ser construída em conjunto.

- A participação durante as aulas é fundamental;
- O processo de aprendizagem depende de aproximadamente **50% do professor e 50% do aluno**;
- Não se aprende programação apenas assistindo outras pessoas programarem;
- Preparem seus computadores para acompanhar as atividades práticas durante a aula;
- Estarei disponível para esclarecer dúvidas e auxiliar durante o desenvolvimento das atividades.

Combinado da turma

Perguntem, participem, pratiquem e não deixem as dúvidas se acumularem.

Desenvolvimento Web com .NET

A disciplina apresenta os principais conceitos e tecnologias utilizados no desenvolvimento de aplicações web modernas com a plataforma .NET.

Conteúdo programático da disciplina:

- Aplicação web utilizando ASP.NET Core MVC e Razor;
- Persistência de dados com Entity Framework Core;
- Desenvolvimento de uma API REST;
- Consumo da API com JavaScript, Fetch e React;
- Autenticação e autorização;
- Observabilidade e resiliência;
- Build, implantação e práticas de CI/CD.

Avaliações da Disciplina

A nota de cada avaliação será composta por duas partes:

- **Trabalho:** Peso de **20%** da nota;
- **Prova:** Peso de **80%** da nota.

Composição da nota

$$\text{Nota} = (\text{Trabalho} \times 0,2) + (\text{Prova} \times 0,8)$$

Avaliação	Data
NP1	23/09/2026
NP2	11/11/2026

Conceitos

O que é uma aplicação Web?

Uma aplicação web é um software executado em um servidor e acessado por meio de um navegador.

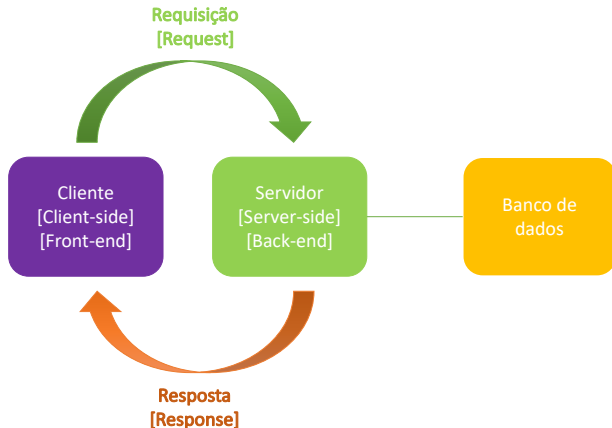
Diferentemente das aplicações desktop, que normalmente precisam ser instaladas no computador, as aplicações web podem ser utilizadas em diferentes dispositivos conectados à internet.

Exemplos de aplicações web:

- Redes sociais, como Facebook, Instagram e X;
- Serviços de e-mail, como Gmail e Outlook;
- Comércio eletrônico, como Amazon e Mercado Livre;
- Sistemas de gestão, como ERPs, CRMs, Trello e Notion;
- Plataformas de streaming, como Netflix, Spotify e YouTube.

Arquitetura de uma aplicação Web

Uma aplicação web funciona por meio da comunicação entre o cliente e o servidor. O cliente envia uma requisição, o servidor processa os dados, pode acessar o banco de dados e, em seguida, retorna uma resposta ao usuário.



O *front-end* é a parte da aplicação web que o usuário visualiza e com a qual interage diretamente.

Ele é responsável por:

- Apresentar informações ao usuário;
- Exibir páginas, menus e formulários;
- Receber dados informados pelo usuário;
- Realizar validações na interface;
- Enviar requisições ao servidor;
- Apresentar as respostas recebidas do back-end.

O código do front-end é executado principalmente no navegador do usuário.

HTML - HyperText Markup Language

Linguagem de marcação responsável por definir a estrutura das páginas web, organizando textos, imagens, formulários, tabelas e links.

CSS - Cascading Style Sheets

Linguagem utilizada para definir a aparência das páginas, incluindo cores, fontes, espaçamentos, layouts e responsividade.

Frameworks e bibliotecas CSS: Bootstrap, Tailwind CSS, Foundation e Bulma.

JavaScript

Linguagem de programação utilizada para adicionar interatividade, manipular elementos da página e realizar comunicação com o servidor.

Bibliotecas e frameworks JavaScript: React, Angular, Vue.js e jQuery.

Servidor (Back-end)

O *back-end* é a parte da aplicação web executada no servidor.

Ele é responsável por:

- Receber e processar requisições;
- Executar regras de negócio;
- Validar os dados recebidos;
- Realizar autenticação e autorização;
- Acessar e modificar informações no banco de dados;
- Gerar respostas para o cliente.

Normalmente, o usuário não interage diretamente com o código do back-end.

Linguagens de programação

C#, Java, Python, PHP, JavaScript e Ruby são exemplos de linguagens utilizadas para implementar aplicações no servidor.

Frameworks

Fornecem recursos e estruturas que facilitam o desenvolvimento de aplicações web.

- ASP.NET Core - C#;
 - Spring Boot - Java;
 - Django - Python;
 - Express - Node.js;
 - Laravel - PHP.
-

Bancos de dados

São utilizados para armazenar e consultar as informações da aplicação.

Exemplos: SQL Server, PostgreSQL, MySQL, SQLite e MongoDB.

Comparativo Front-end e Back-end

Back-end	Front-end
Executado no servidor.	Executado no dispositivo do usuário, geralmente no navegador.
Responsável pelas regras de negócio, processamento e acesso aos dados.	Responsável pela interface, apresentação das informações e interação com o usuário.
Pode acessar bancos de dados e serviços externos.	Envia requisições ao servidor e apresenta as respostas recebidas.
Exemplos: C#, Java, Python, PHP e Node.js.	Exemplos: HTML, CSS, JavaScript e React.

Enquanto o front-end cuida da interface e da interação com o usuário, o back-end é responsável pelo processamento, pelas regras da aplicação e pelo gerenciamento dos dados.

O que é HTTP?

O **HTTP - HyperText Transfer Protocol** é o protocolo utilizado para a comunicação entre clientes e servidores na Web.

Por meio do HTTP, o cliente envia uma **requisição** e o servidor retorna uma **resposta**.

Cliente → Requisição HTTP → Servidor

Cliente ← Resposta HTTP ← Servidor

O HTTP define como as mensagens são organizadas, mas não determina como a aplicação processará os dados.

Requisição e Resposta HTTP

Uma mensagem HTTP possui informações que permitem identificar a operação solicitada e o resultado obtido.

Requisição

- Método HTTP;
- Endereço ou rota;
- Cabeçalhos;
- Corpo da requisição, quando necessário.

GET /produtos

Resposta

- Código de status;
- Cabeçalhos;
- Conteúdo retornado;
- HTML, JSON, imagem ou arquivo.

200 OK

Principais Métodos

- **GET**: Consultar dados;
- **POST**: Enviar ou cadastrar dados;
- **PUT**: Atualizar um recurso;
- **DELETE**: Remover um recurso.

Faixas dos Códigos de Status HTTP

- **1xx – Informativos**: a requisição foi recebida e o processamento continua;
- **2xx – Sucesso**: a requisição foi recebida, entendida e processada com sucesso;
- **3xx – Redirecionamento**: alguma ação adicional é necessária para completar a requisição;
- **4xx – Erro do cliente**: existe algum problema relacionado à requisição enviada pelo cliente;
- **5xx – Erro do servidor**: o servidor encontrou um problema ao processar a requisição.

Exemplo de GET

Requisição

```
GET /produtos/10 HTTP/1.1
Host: localhost:5001
Accept: application/json
```

Resposta

```
HTTP/1.1 200 OK
Content-Type: application/json
{"id":10,"nome":"Teclado","preco":150}
```

Exemplo de POST

Requisição

```
POST /produtos HTTP/1.1
Host: localhost:5001
Content-Type: application/json
{"nome":"Mouse","preco":80}
```

Resposta

```
HTTP/1.1 201 Created
Content-Type: application/json
{"id":11,"nome":"Mouse","preco":80}
```

HTTP e HTTPS

O **HTTPS - HyperText Transfer Protocol Secure** é a versão segura do HTTP.

No HTTPS, a comunicação entre o cliente e o servidor é protegida por criptografia utilizando TLS.

HTTP	HTTPS
Comunicação sem criptografia.	Comunicação protegida por criptografia.
Utiliza normalmente a porta 80.	Utiliza normalmente a porta 443.
Os dados podem ser interceptados e lidos.	Os dados transmitidos são protegidos.

O que o HTTPS protege?

O HTTPS utiliza certificados digitais e o protocolo TLS para oferecer três propriedades importantes:

- **Confidencialidade:** Dificulta que terceiros leiam os dados transmitidos;
- **Integridade:** Permite verificar se os dados foram alterados durante a transmissão;
- **Autenticidade:** Ajuda a confirmar que o usuário está se comunicando com o servidor correto.

O HTTPS protege a comunicação, mas não elimina falhas de segurança existentes na aplicação.

.NET, ASP.NET Core e Razor

O que é .NET?

O **.NET** é uma plataforma de desenvolvimento criada pela Microsoft para construir e executar diferentes tipos de aplicações.

Ele fornece bibliotecas, ferramentas e um ambiente de execução que permitem desenvolver aplicações para Web, desktop, dispositivos móveis, nuvem e outros cenários.

Linguagens utilizadas no .NET:

- **C#**: principal linguagem da plataforma e utilizada na disciplina;
- **F#**: linguagem com forte suporte ao paradigma funcional;
- **Visual Basic .NET**: linguagem tradicional da plataforma Microsoft.

Em resumo

O .NET não é uma linguagem de programação. Ele é a plataforma na qual linguagens como C#, F# e Visual Basic podem ser utilizadas para desenvolver aplicações.

O que é ASP.NET Core?

O **ASP.NET Core** é o framework da plataforma .NET utilizado para o desenvolvimento de aplicações Web.

Ele fornece recursos para:

- Receber e processar requisições HTTP;
- Configurar rotas e endpoints;
- Desenvolver aplicações com MVC;
- Desenvolver páginas com Razor;
- Construir APIs REST;
- Acessar bancos de dados;
- Implementar autenticação e autorização;
- Configurar logging, segurança e publicação.

O que é Razor?

O **Razor** é um mecanismo de criação de páginas dinâmicas utilizado no ASP.NET Core.

Ele permite combinar:

- A estrutura visual da página, escrita em HTML;
- Os dados produzidos pela aplicação;
- Pequenas instruções escritas em C#.

Principal finalidade

Gerar páginas HTML dinamicamente no servidor, de acordo com os dados e com o estado da aplicação.

O navegador não interpreta Razor ou C#. Ele recebe apenas o HTML resultante do processamento realizado no servidor.

Exemplo de código Razor

O Razor utiliza o caractere @ para indicar trechos de código C# dentro de uma página HTML.

Arquivo Index.cshtml

```
@{  
    string nome = "Lucas";  
    int quantidade = 3;  
}  
  
<h1>Bem-vindo, @nome!</h1>  
  
<p>Quantidade de produtos: @quantidade</p>
```

Nesse exemplo, o servidor processa o código C# e envia ao navegador apenas o HTML resultante.

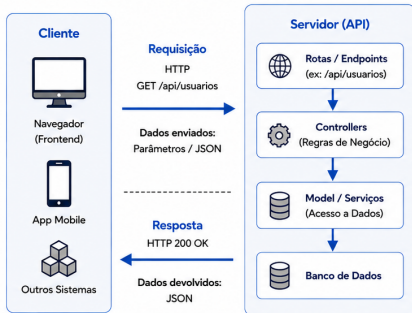
Como as tecnologias se relacionam?

Tecnologia	Responsabilidade
C#	Linguagem utilizada para programar a aplicação.
.NET	Plataforma que fornece bibliotecas e o ambiente de execução.
ASP.NET Core	Framework utilizado para desenvolver aplicações Web e APIs.
Razor	Sintaxe utilizada para criar páginas HTML dinâmicas com C#.

Dois Modelos de Aplicações Web

1) API (Web Services / REST)

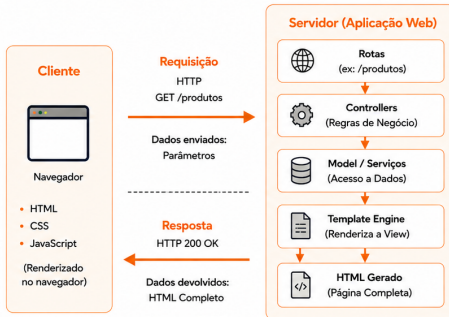
Aplicação focada em fornecer dados e serviços



- O servidor devolve apenas os dados (JSON, XML, etc.).
- A renderização e exibição ficam a cargo do cliente.

2) Template Engine (Aplicação Web Server-Side)

Aplicação que renderiza HTML no servidor e entrega páginas



- O servidor monta a página (HTML) usando os dados e o template.
- O navegador recebe o HTML pronto para exibir.

MVC

O que é MVC?

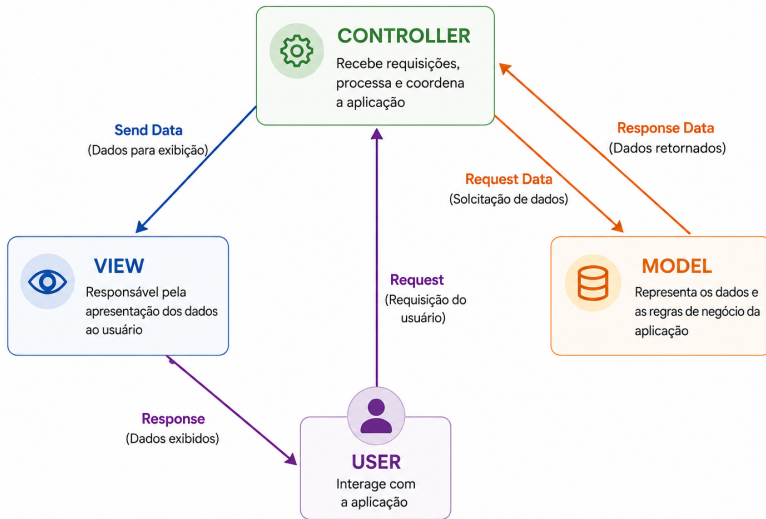
O **MVC** — **Model, View e Controller** é um padrão arquitetural utilizado para organizar aplicações, separando responsabilidades.

- **Model:** Representa os dados e as regras da aplicação;
- **View:** Representa a interface apresentada ao usuário;
- **Controller:** Recebe as requisições, coordena o processamento e define a resposta.

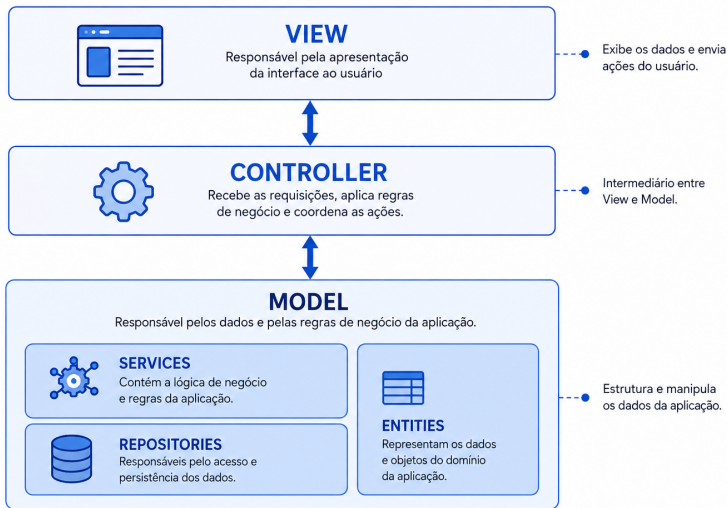
Principal objetivo

Separar a interface, os dados e o controle da aplicação, facilitando a organização, a manutenção e a evolução do sistema.

Arquitetura MVC



Arquitetura MVC



Criação do projeto Web

Para iniciar a aplicação, utilizaremos o modelo de projeto:

ASP.NET Core Web App Model-View-Controller

Configurações utilizadas:

- Framework: **.NET 9**;
- Linguagem: **C#**;
- Arquitetura inicial: **MVC**;
- Autenticação: nenhuma, neste primeiro momento;
- HTTPS habilitado.

Objetivo da prática

Criar, executar e conhecer a estrutura inicial de uma aplicação Web com ASP.NET Core.

Primeiro Projeto

Estrutura inicial do projeto

Ao criar uma aplicação ASP.NET Core MVC, o projeto é organizado em pastas e arquivos com responsabilidades específicas.

- **Controllers:** Recebem as requisições e controlam o fluxo da aplicação;
- **Models:** Representam os dados e as regras utilizadas pelo sistema;
- **Views:** Contêm as páginas Razor exibidas ao usuário;
- **wwwroot:** Armazena arquivos públicos, como CSS, JavaScript e imagens;
- **Properties:** Reúne configurações utilizadas durante a execução local.

Essa organização ajuda a separar as responsabilidades e facilita a manutenção do projeto.

Arquivos principais do projeto

- **Program.cs:** Ponto inicial da aplicação. Nele são registrados os serviços, configurado o pipeline HTTP e definido o roteamento da aplicação;
- **appsettings.json:** Armazena configurações da aplicação, como níveis de log, strings de conexão e parâmetros utilizados pelo sistema;
- **launchSettings.json:** Contém configurações usadas durante a execução local, como endereços, portas, ambiente e perfis de inicialização.

Importante

O `launchSettings.json` é voltado principalmente ao ambiente de desenvolvimento, enquanto o `appsettings.json` armazena configurações da aplicação.

A **semântica do HTML** consiste em utilizar elementos que descrevem o **significado e a função** de cada parte da página, e não apenas sua aparência.

- `<header>` – Cabeçalho da página ou seção;
- `<nav>` – Área destinada à navegação;
- `<main>` – Conteúdo principal da página;
- `<footer>` – Rodapé da página ou seção.

Por que utilizar HTML semântico?

Facilita a organização e manutenção do código, além de melhorar a acessibilidade e a interpretação da página por navegadores e mecanismos de busca.

Estrutura do _Layout.cshtml

O arquivo `_Layout.cshtml` define a estrutura comum utilizada pelas diferentes páginas da aplicação.

```
<body>
  <header>
    <nav>
      ...
    </nav>
  </header>

  <main>
    @RenderBody()
  </main>

  <footer>
    ...
  </footer>
</body>
```

- header: Cabeçalho da aplicação;
- nav: Área de navegação;
- main: Conteúdo principal;
- footer: Rodapé compartilhado.

O papel do @RenderBody()

O `@RenderBody()` representa o ponto do layout onde será inserido o conteúdo da **View atual**.

```
1 <main>
2   @RenderBody()
3 </main>
```

Ideia principal

O `_Layout.cshtml` mantém a estrutura comum da aplicação, enquanto cada **View** fornece apenas o conteúdo específico da página.

Exemplo:

- `Index.cshtml` fornece o conteúdo da página inicial;
- `Clientes/Index.cshtml` fornece o conteúdo da página de clientes;
- Ambos podem utilizar o mesmo `_Layout.cshtml`.

Prática: Criando um Model

Nesta atividade, criaremos uma página que apresenta os dados de um produto.

- ❶ Criar a classe Produto na pasta Models;
- ❷ Instanciar um produto no HomeController;
- ❸ Enviar o objeto para a View;
- ❹ Criar uma View fortemente tipada;
- ❺ Exibir os dados utilizando o objeto Model;
- ❻ Adicionar a página ao menu da aplicação.

Obrigado!

Dúvidas?